Thala LSD

Security Assessment

February 14th, 2025 — Prepared by OtterSec

Andreas Mantzoutas andreas@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-LSD-ADV-00 | Inflation Attack on Zero Total Stake 6

OS-LSD-ADV-01 | Lack of Token Unfreeze Functionality 7

OS-LSD-ADV-02 | Inconsistency in Maintaining One-to-One Peg 8

General Findings **9**

OS-LSD-SUG-00 | Inconsistent Fee Exemption Handling 10

Appendices

Vulnerability Rating Scale **11**

Procedure **12**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Thala Protocol engaged OtterSec to assess the `thala-lsd` program. This assessment was conducted between January 30th and February 6th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 4 findings throughout this audit engagement.

In particular, the staking of `thAPT` is vulnerable to inflation attacks if the total stake approaches zero. (OS-LSD-ADV-00). Additionally, the staking module permanently freezes token stores without any corresponding method to unfreeze them (OS-LSD-ADV-01). Furthermore, the one-to-one peg between `thAPT` and staked `APT` may break due to the unrestricted burning, and while the reconcile functionality may mint `thAPT` to restore the peg, it also enables arbitrary supply manipulation (OS-LSD-ADV-02).

We also made a suggestion to ensure that the `thAPT` staking functionality takes into consideration the fee exempt accounts (OS-LSD-SUG-00).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/ThalaLabs/thala-modules. This audit was performed against commit b60ea86.

A brief description of the program is as follows:

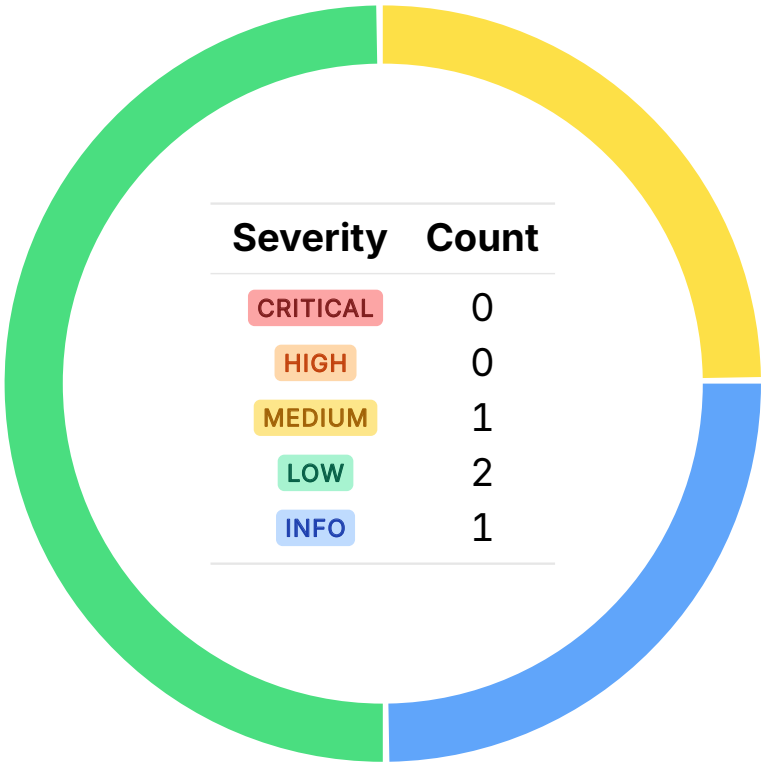
Name	Description
thala-lsd	A liquid staking derivative allowing staking tokens to be liquidated through `thAPT`.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 4 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	1
LOW	2
INFO	1

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-LSD-ADV-00	MEDIUM	RESOLVED	`stake_thAPT_v2` is vulnerable to inflation attacks if the total stake approaches zero.
OS-LSD-ADV-01	LOW	RESOLVED	`freeze_thapt_coin_stores` permanently freezes token stores without any method to unfreeze them.
OS-LSD-ADV-02	LOW	RESOLVED	The `1:1` peg between `thAPT` and staked `APT` may break due to the unrestricted burning `burn_from_thapt`, and while `reconcile` may mint `thAPT` to restore the peg, it also enables arbitrary supply manipulation.

© 2025 Otter Audits LLC. All Rights Reserved.

Inflation Attack on Zero Total Stake MEDIUM OS-LSD-ADV-00

Description

`staking::stake_thAPT_v2` is susceptible to an inflation attack, which may allow the first depositor to exploit subsequent depositors by manipulating the exchange rate. This can be achieved by making an initial deposit, which would depeg the 1:1 initial ratio between the `sthAPT_supply` and the `thAPT_staking` amount due to the staking fee. After this point, the attacker can continue making progressively larger deposits into the pool, resulting in zero minted `sthAPT`, further inflating the price.

>_ *thala_lsd/sources/staking.move* RUST

```rust
public fun stake_thAPT_v2(coin: Coin<ThalaAPT>): Coin<StakedThalaAPT> acquires TLSD, PauseFlag {
    [...]
    // exchange_rate = thAPT_staking / sthAPT_supply
    // sthAPT_amount = thAPT_amount / exchange_rate = thAPT_amount * sthAPT_supply /
        ↪  thAPT_staking
    let (thAPT_staking, sthAPT_supply) = thAPT_sthAPT_exchange_rate();
    let sthAPT_amount = math64::mul_div(thAPT_amount - fee_amount, sthAPT_supply,thAPT_staking);
    [...]
}
```

The vulnerability revolves around calculating `sthAPT_amount` for successive deposits. Due to the initial supply and a substantial donation, the value of `sthAPT_amount` for subsequent deposits may be truncated to zero if the depositor attempts to deposit an amount lower than the share value. This creates an unintended and potentially exploitable scenario where subsequent depositors may not obtain the anticipated amount of minted shares. Moreover, this problem may be exploited in coordination with a front-running attack, wherein the attacker strategically times a substantial token donation just before the second deposit to maximize the impact of the flooring issue. However, in order to inflate the share price effectively, the exploiter needs to be the sole owner of shares, which usually requires an empty pool. Thus, in the current deployment, unless the total stake goes to zero, this will not occur. However, this issue may come into effect when deploying in a new environment.

Remediation

Permanently lock a portion of the initial deposit to prevent any depositor from becoming the sole owner of the pool shares. Additionally, establish a mechanism to ensure that the minted amount is never zero.

Patch

Fixed in PR#911.

© 2025 Otter Audits LLC. All Rights Reserved.

Lack of Token Unfreeze Functionality LOW OS-LSD-ADV-01

Description

The current implementation of `staking` has functionality to freeze the coin stores of `thAPT` tokens for a list of specified account addresses via `freeze_thapt_coin_stores`. However, there is an absence of a corresponding unfreeze function which may result in a loss of liquidity as users will be unable to access their tokens preventing them from utilizing them.

>_ *thala_lsd/sources/staking.move* RUST

```rust
public entry fun freeze_thapt_coin_stores(manager: &signer, account_addresses: vector<address>)
    ↪  acquires TLSD {
    assert!(manager::is_authorized(manager), ERR_TLSD_UNAUTHORIZED);

    let freeze_cap =
        ↪  &borrow_global<TLSD>(package::resource_account_address()).thAPT_freeze_capability;
    vector::for_each(account_addresses, |account_address| {
        coin::freeze_coin_store(account_address, freeze_cap);
    })
}
```

Remediation

Introduce a function such as `unfreeze_thapt_coin_stores` that reverses the freezing action.

Patch

Fixed in 5ba884a.

© 2025 Otter Audits LLC. All Rights Reserved.

Inconsistency in Maintaining One-to-One Peg LOW OS-LSD-ADV-02

Description

In `staking`, `burn_from_thapt` and `reconcile` introduce risks that may break the intended `1:1` peg between `thAPT` and staked `APT`. `thAPT` is meant to be pegged `1:1` to staked `APT`, representing a claim on the underlying collateral. However, by burning `thAPT`, the total supply decreases while the underlying `APT` remains unchanged.

>_ *thala_lsd/sources/staking.move* RUST

```rust
public entry fun burn_from_thapt(manager: &signer, account_address: address, amount: u64)
    ↪  acquires TLSD {
    assert!(manager::is_authorized(manager), ERR_TLSD_UNAUTHORIZED);
    coin::burn_from(account_address, amount,
        ↪  &borrow_global<TLSD>(package::resource_account_address()).thAPT_burn_capability)
}

public entry fun reconcile(manager: &signer, amount: u64) acquires TLSD {
    assert!(manager::is_authorized(manager), ERR_TLSD_UNAUTHORIZED);
    let minted = coin::mint(amount,
        ↪  &borrow_global<TLSD>(package::resource_account_address()).thAPT_mint_capability);
    coin::deposit(signer::address_of(manager), minted)
}
```

While this may be mitigated via `reconcile`, which allows the authorized manager to mint new `thAPT` tokens, excessive minting may inflate the supply of `thAPT`, breaking the peg in the opposite direction (more `thAPT` in circulation than staked `APT`) giving rise to a similar problem.

Remediation

Introduce checks to ensure the total `thAPT` supply always matches the total staked `APT`.

Patch

Fixed in 5ba884a.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-LSD-SUG-00	`stake_thAPT_v2` does not check for fee exemptions, unlike `request_unstake_APT`, resulting in inconsistent fee application for whitelisted accounts.

© 2025 Otter Audits LLC. All Rights Reserved.

Inconsistent Fee Exemption Handling OS-LSD-SUG-00

Description

There is an inconsistency in the fee handling logic in `staking::stake_thAPT_v2`. In `stake_thAPT_v2`, all users, regardless of whether they are in the `FeesExempt` whitelist, are charged the staking fee. Thus, it does not account for `fee_exempt` accounts. In `request_unstake_APT` on the other hand, the fee is calculated only if the user is not in the whitelist.

Remediation

Update the fee calculation logic in `stake_thAPT_v2` to check for fee exemptions.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.